Frontend guide: editing staff profiles

This guide covers the backend behavior and API calls for adding profile editing to the existing product.

What users can edit

All profile edits use the authenticated session cookie and a matching `Origin` header. The API returns `403 permission_denied` when the signed-in user is not allowed to change the requested profile or fields.

Caller	Allowed profile edits
Staff member	Their own full name and work email
Admin or Super Admin	Any staff profile's full name, work email, job title, and department
Staff member editing someone else	Not allowed

`job_title` and `department_id` are organization-managed fields. Staff can also update their own profile image through the separate profile-picture endpoint. An Admin or Super Admin with `staff.update` can update another user's profile image.

The staff number (`staff_id`), linked account (`account_id`), and employment status are not editable through profile editing. An empty `work_email` can be sent as `null` to clear it.

Update a profile

Request

```
PATCH /api/v1/staff/{staffId}
```

```
Content-Type: application/json



Origin: https://your-product.example



Cookie: mothercat_session=<session>
```

`staffId` is the staff record's UUID (`staff.id`), not the human-readable `staff_id` number. Send only fields the user changed; fields omitted from the body stay unchanged.

Staff self-service example:

```
{


  "full_name": "Ada Okafor",


  "work_email": "ada.okafor@example.test"


}
```

Admin/Super Admin example:

```
{
```

```
  "full_name": "Ada Okafor",

  "work_email": null,

  "job_title": "Senior Site Engineer",

  "department_id": "1c9491b8-21ce-4c66-a025-006bb346b810"

}
```

Success response (`200 OK`)

```
{

  "staff": {

    "id": "7da27cda-3496-4d02-9d99-fb0e956297e5",

    "staff_id": "23847",

    "full_name": "Ada Okafor",
```

```
    "work_email": "ada.okafor@example.test",




    "job_title": "Senior Site Engineer",




    "department_id": "1c9491b8-21ce-4c66-a025-006bb346b810",




    "profile_image_url": null,




    "employment_status": "active"




  }



}
```

The server may return additional nullable fields, such as `account_id`. Use the returned `staff` object to refresh the profile in the UI.

For browser requests, include the session cookie:

```
const response = await fetch(`/api/v1/staff/${staff.id}`, {




  method: "PATCH",
```

```
  credentials: "include",

  headers: { "Content-Type": "application/json" },

  body: JSON.stringify(changedFields),

});

const result = await response.json();

if (!response.ok) {

  // Show the API error to the user; do not treat a failed save as
successful.

  throw new Error(result.error?.code ?? "profile_update_failed");

}
```

```
setStaff(result.staff);
```

Use the application's existing API client if it already handles session credentials and errors. The backend checks that the browser's `Origin` matches the API host, so keep this request on the normal authenticated application flow.

Upload a profile picture

Use the existing endpoint separately from the JSON profile update:

```
POST /api/v1/staff/{staffId}/profile-picture

Content-Type: multipart/form-data
```

Submit one multipart file field named `file`. Supported formats are JPEG, PNG, and GIF; the maximum file size is 5 MiB. The response is the same `{"staff": {...}}` shape. Refresh the profile from the returned object after a successful upload.

Error handling

Status	Meaning	Suggested frontend behavior
`400`	Invalid field, email, staff UUID, or request body	Keep the form open and show field/input errors
`40`	Session missing or expired	Start the existing

Status	Meaning	Suggested frontend behavior
1		sign-in flow
403	Not allowed to edit this profile or field, or invalid origin	Show a permission message; do not retry unchanged
404	Staff record does not exist	Refresh the directory or show a not-found state
503	Required backend or audit service unavailable	Show a retryable save error
500	Unexpected backend failure	Show a retryable save error and preserve form values

The backend audits successful update requests by field name only; it does not put submitted personal values into the audit details.

Suggested UI behavior

1. Reuse the current staff-profile view and show an Edit action for the signed-in user's own record. 2. Show name and work email in the self-service form. Show job title and department controls only to Admin/Super Admin users. 3. Use the existing role/permission state for UI visibility, but treat the API response as authoritative; never rely on hidden controls for authorization. 4. Disable Save while a request is pending. On success, replace the local profile with `result.staff`; on failure, retain the user's edits. 5. Keep profile-picture upload as its own multipart request.